14BHD Informatica – Lab04

Esercitazione di Laboratorio 4

Temi trattati

- **Ciclo `for` su una stringa:** scorrere i caratteri di una stringa uno alla volta, senza gestire un indice.
- **Ciclo `for` con `range`:** ripetere un blocco di istruzioni un numero di volte noto in anticipo, con `range(fine)`, `range(inizio, fine)` e `range(inizio, fine, passo)`, anche con un passo negativo. Il valore `fine` è sempre escluso.
- **`enumerate`:** avere a ogni iterazione sia la posizione sia il valore di un elemento.
- **`break` e `continue`:** interrompere un ciclo prima della fine, quando il risultato è già noto, oppure saltare il resto di un'iterazione e passare subito alla successiva.
- **Cicli annidati:** un ciclo dentro un altro, ad esempio per esaminare ogni numero di un intervallo e, per ciascuno, ogni suo possibile divisore.
- **f-string:** inserire valori ed espressioni in una stringa e controllarne il formato: ampiezza, numero di decimali, allineamento, zeri a sinistra, percentuali, basi diverse dalla 10.

Nel Lab03 tutti i cicli erano `while`. Quando il numero di iterazioni è noto prima di iniziare (le lettere di una stringa, i numeri di un intervallo) il ciclo `for` è più semplice e più sicuro: non c'è un contatore da inizializzare e aggiornare a mano, e quindi non c'è il rischio di dimenticarlo.

L'esercitazione è divisa in due parti. Gli **esercizi in laboratorio** (1–5) sono quelli da cui partire: l'esercizio 1 lo risolviamo insieme, passo per passo, e serve da traccia per gli altri. Gli **esercizi aggiuntivi** (6–9) ricalcano gli stessi temi e sono consigliati dopo aver finito quelli «normali».

Esercizi in laboratorio

1 L'orario della navetta

Svolto insieme in aula – `for` con `range`, f-string, `enumerate`, `continue` e `break`

Una navetta collega la stazione al Politecnico. La prima corsa parte alle 7:30, l'ultima alle 19:00, e tra una corsa e la successiva passano sempre 30 minuti. Vogliamo stampare l'orario delle corse e dire a chi arriva alla fermata quando passa la prossima. Le corse sono note prima di iniziare: è il caso tipico del ciclo `for`.

1.1 Stampate gli orari di tutte le corse, uno per riga, nel formato `hh:mm`, con due cifre sia per le ore sia per i minuti.

Tip: conviene contare il tempo in minuti dalla mezzanotte, come nel Lab02 (esercizio B.5): le 7:30 sono `7 * 60 + 30 = 450` minuti. `range(inizio, fine, passo)` produce i numeri

da `inizio` a `fine`, *escluso* `fine`, a passi di `passo`. Per tornare a ore e minuti servono `//` e `%`. Nelle f-string lo specificatore `02d` stampa un intero su due cifre, con uno zero a sinistra se serve: `f"{7:02d}"` dà `07`.

Esempio (le righe intermedie sono omesse):

```
07:30
08:00
08:30
...
19:00
```

Soluzione costruita insieme:

```
PRIMA_CORSA = 7 * 60 + 30      # 07:30 in minuti dalla mezzanotte
ULTIMA_CORSA = 19 * 60         # 19:00
INTERVALLO = 30                # minuti tra una corsa e la successiva

for t in range(PRIMA_CORSA, ULTIMA_CORSA + 1, INTERVALLO):
    ore = t // 60
    minuti = t % 60
    print(f"{ore:02d}:{minuti:02d}")
```

Il secondo argomento di `range` è escluso: per avere anche la corsa delle 19:00 serve `ULTIMA_CORSA + 1`. Con il `while` del Lab03 il contatore andava inizializzato prima del ciclo e aggiornato a ogni iterazione; con il `for` se ne occupa `range`: `t` vale 450, poi 480, 510, ... fino a 1140.

1.2 Numerate le corse: davanti all'orario stampate il numero della corsa, partendo da 1, su due caratteri, in modo che gli orari restino allineati.

Tip: `enumerate` restituisce a ogni iterazione una coppia `(indice, valore)` e funziona con qualsiasi contenitore, anche con un `range`. L'indice parte da 0. Dentro le graffe di una f-string si può scrivere un'espressione, non solo una variabile: `{i + 1:2d}`.

Esempio (le righe intermedie sono omesse):

```
Corsa  1 - 07:30
Corsa  2 - 08:00
Corsa  3 - 08:30
...
Corsa 24 - 19:00
```

Soluzione costruita insieme:

```
PRIMA_CORSA = 7 * 60 + 30
ULTIMA_CORSA = 19 * 60
INTERVALLO = 30

for (i, t) in enumerate(range(PRIMA_CORSA, ULTIMA_CORSA + 1, INTERVALLO)):
    print(f"Corsa {i + 1:2d} - {t // 60:02d}:{t % 60:02d}")
```

1.3 Dalle 13:00 alle 14:00 l'autista è in pausa pranzo: le corse delle 13:00, delle 13:30 e delle 14:00 non partono. Stampate l'orario senza queste corse, usando `continue`. Le corse rimaste vanno numerate senza buchi.

Primo tentativo, con `enumerate`*:*

```
PRIMA_CORSA = 7 * 60 + 30
ULTIMA_CORSA = 19 * 60
INTERVALLO = 30
PAUSA_INIZIO = 13 * 60
PAUSA_FINE = 14 * 60

for (i, t) in enumerate(range(PRIMA_CORSA, ULTIMA_CORSA + 1, INTERVALLO)):
    if t >= PAUSA_INIZIO and t <= PAUSA_FINE:
        continue
    print(f"Corsa {i + 1:2d} - {t // 60:02d}:{t % 60:02d}")
```

Esempio (le righe intermedie sono omesse):

```
...
Corsa 10 - 12:00
Corsa 11 - 12:30
Corsa 15 - 14:30
Corsa 16 - 15:00
...
```

Non funziona del tutto: le corse della pausa non vengono stampate, ma `enumerate` conta tutte le iterazioni, anche quelle interrotte da `continue`. Dopo la corsa 11 viene la 15.

Soluzione costruita insieme, con un contatore:

```
PRIMA_CORSA = 7 * 60 + 30
ULTIMA_CORSA = 19 * 60
INTERVALLO = 30
PAUSA_INIZIO = 13 * 60        # dalle 13:00...
PAUSA_FINE = 14 * 60          # ...alle 14:00 comprese, nessuna corsa

corsa = 0
for t in range(PRIMA_CORSA, ULTIMA_CORSA + 1, INTERVALLO):
    if t >= PAUSA_INIZIO and t <= PAUSA_FINE:
        continue              # salta la corsa: si passa al t successivo
    corsa = corsa + 1
    print(f"Corsa {corsa:2d} - {t // 60:02d}:{t % 60:02d}")
```

Esempio (le righe intermedie sono omesse):

```
...
Corsa 10 - 12:00
Corsa 11 - 12:30
Corsa 12 - 14:30
Corsa 13 - 15:00
...
```

`continue` salta il resto dell'iterazione: per le corse della pausa il contatore `corsa` non viene incrementato e la `print` non viene eseguita. Il ciclo riprende con il valore successivo di `t`.

1.4 Chiedete all'utente l'orario in cui arriva alla fermata, nel formato `hh:mm` (supponete che sia scritto correttamente, con due cifre per le ore e due per i minuti), e stampate la prima corsa che può prendere e quanti minuti dovrà aspettare. Appena trovata la corsa il ciclo non deve proseguire: usate `break`. Se l'utente arriva dopo l'ultima corsa, stampate un messaggio.

Tip: per estrarre ore e minuti usate lo slicing, come nel Lab02. Il ciclo ora può finire in due modi: con `break`, se la corsa c'è, oppure perché il `range` è finito, se non c'è. Come nel Lab03 (esercizio 5.3), dopo il ciclo serve una decisione per capire quale dei due casi si è verificato: una variabile booleana lo ricorda.

Esempio (tre esecuzioni):

```
A che ora arrivi alla fermata (hh:mm)? 10:05
Prima corsa utile: 10:30 (attesa 25 minuti)

A che ora arrivi alla fermata (hh:mm)? 13:10
Prima corsa utile: 14:30 (attesa 80 minuti)

A che ora arrivi alla fermata (hh:mm)? 19:20
Nessuna corsa: l'ultima parte alle 19:00
```

Soluzione costruita insieme:

```
PRIMA_CORSA = 7 * 60 + 30
ULTIMA_CORSA = 19 * 60
INTERVALLO = 30
PAUSA_INIZIO = 13 * 60
PAUSA_FINE = 14 * 60

arrivo = input("A che ora arrivi alla fermata (hh:mm)? ")
t_arrivo = int(arrivo[0:2]) * 60 + int(arrivo[3:5])

trovata = False
for t in range(PRIMA_CORSA, ULTIMA_CORSA + 1, INTERVALLO):
    if t >= PAUSA_INIZIO and t <= PAUSA_FINE:
        continue
    if t >= t_arrivo:
        trovata = True
        break                  # inutile guardare le corse successive

if trovata:
    orario = f"{t // 60:02d}:{t % 60:02d}"
    print(f"Prima corsa utile: {orario} (attesa {t - t_arrivo} minuti)")
else:
    ultima = f"{ULTIMA_CORSA // 60:02d}:{ULTIMA_CORSA % 60:02d}"
    print(f"Nessuna corsa: l'ultima parte alle {ultima}")
```

Dopo `break` la variabile `t` conserva il valore che aveva quando il ciclo si è interrotto: è proprio l'orario della corsa trovata. Notate che `continue` e `break` convivono nello stesso ciclo: le corse della pausa vengono saltate anche durante la ricerca.

2 Il filamento di DNA

Ciclo `for` su una stringa, f-string

Il DNA si può rappresentare come una stringa di quattro lettere, le iniziali delle basi azotate: A (adenina), C (citosina), G (guanina) e T (timina). Nei primi punti analizzate il filamento

```
DNA = "CCGTAGCATGGCTTACGATTGACCGTATGCAAGTCG"
```

che definite come costante, scorrendolo con un ciclo `for` una lettera alla volta.

Tip: `for base in DNA` assegna a `base`, una dopo l'altra, tutte le lettere della stringa. Non serve un indice, e quindi non si può sbagliare l'ultimo valore, come poteva capitare con il `while` del Lab03 (esercizio 7).

2.1 Contate quante volte compare ciascuna delle quattro basi e stampate la lunghezza del filamento e i quattro conteggi.

Tip: servono quattro contatori, inizializzati a 0 prima del ciclo.

Esempio:

```
Lunghezza: 36 basi
A: 8
C: 9
G: 10
T: 9
```

2.2 Accanto a ogni conteggio stampate la percentuale sul totale, con una cifra decimale. Stampate infine il *contenuto GC*, cioè la percentuale di basi che sono G oppure C: è un parametro molto usato in biologia, perché un filamento ricco di G e C è più stabile.

Tip: il tipo `%` delle f-string moltiplica il numero per 100 e aggiunge il simbolo di percentuale: `f"{0.25:.1%}"` dà `25.0%`. Con `2d` i conteggi occupano sempre due caratteri e restano incolonnati.

Esempio:

```
Lunghezza: 36 basi
A:  8 (22.2%)
C:  9 (25.0%)
G: 10 (27.8%)
T:  9 (25.0%)
Contenuto GC: 52.8%
```

2.3 Nella doppia elica a ogni base è legata la sua complementare: A con T e C con G. Costruite e stampate il filamento complementare, in cui ogni A diventa T, ogni T diventa A, ogni C diventa G e ogni G diventa C.

Tip: partite da una stringa vuota e aggiungete una lettera a ogni iterazione.

Esempio:

```
Filamento:     CCGTAGCATGGCTTACGATTGACCGTATGCAAGTCG
Complementare: GGCATCGTACCGAATGCTAACTGGCATACGTTCAGC
```

2.4 Ora il filamento lo scrive l'utente. Accettate anche le lettere minuscole e controllate che il filamento contenga soltanto le lettere A, C, G, T: al primo carattere non valido stampate quale carattere è e in che posizione si trova, e fermate il controllo con `break`. Se il filamento è valido, stampate le analisi dei punti precedenti.

Tip: `enumerate` vi dà a ogni iterazione sia la posizione sia la lettera. Per sapere se una lettera è una delle quattro basi potete usare l'operatore `in`: `base in "ACGT"` è vero se `base` è una delle lettere della stringa `"ACGT"`. Il metodo `upper()` converte una stringa in maiuscolo.

Esempio (due esecuzioni):

```
Filamento di DNA: atgcxta
Carattere non valido 'X' in posizione 4

Filamento di DNA: ggatccatgc
Lunghezza: 10 basi
A:  2 (20.0%)
C:  3 (30.0%)
G:  3 (30.0%)
T:  2 (20.0%)
Contenuto GC: 60.0%
Complementare: CCTAGGTACG
```

2.5 Un gene comincia con la tripletta ATG, il *codone di inizio*. Trovate la posizione del primo ATG nel filamento `DNA` del punto 2.1, esaminando una tripletta alla volta: la tripletta che comincia in posizione `i` è `DNA[i:i + 3]`. Fermatevi alla prima che trovate. Qual è l'ultimo valore di `i` che ha senso controllare?

Esempio:

```
Primo codone ATG in posizione 7
```

2.6 **Approfondimento.** Le stringhe hanno già dei metodi che fanno una parte di questo lavoro: `DNA.count("A")` conta le A, e `DNA.find("ATG")` restituisce la posizione della prima tripletta ATG (oppure -1, se non c'è). Con `count` il punto 2.1 si scrive in quattro righe. Quante volte viene scorsa la stringa in quel caso? E con il vostro ciclo?

3 Numeri primi

Ciclo `for` con `break`, cicli annidati

Un numero intero maggiore di 1 è *primo* se è divisibile soltanto per 1 e per se stesso: 2, 3, 5, 7, 11 e 13 sono primi, $9 = 3 \times 3$ no. Per scoprire se un numero non è primo basta trovare un suo divisore: appena se ne trova uno, la risposta è nota e non serve cercarne altri.

3.1 Chiedete un numero intero `N` maggiore di 1 (supponete che lo sia) e stabilite se è primo, cercando un divisore tra 2 e `N - 1`. Appena ne trovate uno fermate il ciclo con `break`, e stampate il divisore trovato.

Tip: `d` è un divisore di `N` se `N % d == 0`. Come nell'esercizio 1.4, il ciclo può finire con `break` oppure perché il `range` è finito: una variabile booleana dice quale dei due casi si è verificato.

Esempio (tre esecuzioni):

```
N: 91
91 non è primo: è divisibile per 7

N: 97
97 è primo

N: 2
2 è primo
```

Perché il programma risponde correttamente anche per `N = 2`? Quante volte viene eseguito il ciclo in quel caso?

3.2 Contate quante divisioni esegue il programma prima di rispondere, e stampatelo. Provate con `N = 1000003`, che è primo. Poi migliorate il programma: se $N = a \times b$ con $a \leq b$, allora a non può superare $\sqrt{N}$, quindi basta cercare i divisori fino a $\sqrt{N}$. Modificate il `range` e confrontate di nuovo il numero di divisioni.

Tip: `math.sqrt` restituisce un `float`, mentre `range` vuole degli interi: convertite con `int()`. E ricordate che la fine del `range` è esclusa.

Esempio (prima e dopo la modifica):

```
N: 1000003
1000003 è primo (divisioni: 1000001)

N: 1000003
1000003 è primo (divisioni: 999)
```

3.3 Stampate tutti i numeri primi minori o uguali a un limite `M` chiesto all'utente, dieci per riga, ciascuno su 5 caratteri, e alla fine quanti sono. Servono due cicli annidati: quello esterno scorre i candidati da 2 a `M`, quello interno cerca un divisore del candidato, come nel punto 3.2.

Tip: `print(..., end="")` stampa senza andare a capo, come nella tabella delle potenze vista a lezione. Contate i primi trovati: quando il contatore è un multiplo di 10, andate a capo con `print()`. Attenzione: `break` interrompe soltanto il ciclo in cui si trova, cioè quello interno; il ciclo esterno prosegue con il candidato successivo.

Esempio:

```
Limite: 100
    2    3    5    7   11   13   17   19   23   29
   31   37   41   43   47   53   59   61   67   71
   73   79   83   89   97
Primi trovati: 25
```

3.4 Due numeri primi si dicono *gemelli* se la loro differenza è 2, come 11 e 13. Stampate le coppie di primi gemelli fino al limite `M`, e quante sono. Quando un candidato non è primo usate `continue`: non c'è altro da fare, e si passa subito al candidato successivo.

Tip: come nel confronto di valori consecutivi visto a lezione, tenete l'ultimo primo trovato in una variabile e confrontatelo con il primo successivo.

Esempio:

```
Limite: 100
(3, 5)
(5, 7)
(11, 13)
(17, 19)
(29, 31)
(41, 43)
(59, 61)
(71, 73)
Coppie di primi gemelli: 8
```

3.5 **Approfondimento.** Il ciclo interno prova tutti i divisori fino a $\sqrt{N}$, anche quelli pari. Se `N` è dispari, ha senso provare 4, 6, 8? Come potete usare il passo di `range` per fare circa metà delle divisioni?

4 Lo scontrino

`for` con `range`, f-string con ampiezza e allineamento, `continue`

La cassa di un negozio stampa lo scontrino: una riga per ogni articolo, con il nome, la quantità, il prezzo unitario e l'importo, in colonne allineate, e il totale in fondo. Prima di iniziare il cassiere dice quanti articoli diversi deve battere: il numero di iterazioni è noto.

4.1 Chiedete il numero di articoli e poi, per ciascuno, il nome, il prezzo unitario e la quantità. Alla fine stampate il totale da pagare, con due decimali.

Tip: `range(1, n + 1)` produce i numeri da 1 a `n`, comodi per numerare gli articoli nelle domande. Anche il testo che `input` mostra all'utente può essere una f-string:

```
input(f"Articolo {i} - nome: ")
```

Esempio:

```
Numero di articoli: 3
Articolo 1 - nome: Pane
Articolo 1 - prezzo: 2.40
Articolo 1 - quantità: 2
Articolo 2 - nome: Latte
Articolo 2 - prezzo: 1.35
Articolo 2 - quantità: 1
Articolo 3 - nome: Mele
Articolo 3 - prezzo: 0.45
Articolo 3 - quantità: 6
Totale: 8.85
```

4.2 Ora stampate lo scontrino vero e proprio, dopo che l'utente ha inserito tutti gli articoli. Ogni riga contiene il nome allineato a sinistra su 12 caratteri, la quantità su 4 caratteri, il prezzo su 8 e l'importo su 9, questi ultimi due con due decimali. Se il nome è più lungo di 12 caratteri, stampatene soltanto i primi 12.

Tip: nelle f-string `<` allinea a sinistra e `>` a destra: `f"{nome:<12}"` occupa sempre almeno 12 caratteri. Le righe vanno stampate alla fine, ma i dati di un articolo si perdono all'iterazione successiva: costruite le righe dentro il ciclo, aggiungendole a una stringa (`scontrino = scontrino + riga`) che stamperete dopo il ciclo. Il carattere speciale `"\n"` va a capo (Lab02, esercizio A.17). Per tagliare il nome usate lo slicing.

Esempio:

```
Numero di articoli: 3
Articolo 1 - nome: Pane
Articolo 1 - prezzo: 2.40
Articolo 1 - quantità: 2
Articolo 2 - nome: Latte
Articolo 2 - prezzo: 1.35
Articolo 2 - quantità: 1
Articolo 3 - nome: Parmigiano reggiano 24 mesi
Articolo 3 - prezzo: 7.90
Articolo 3 - quantità: 1

Articolo     Qtà  Prezzo  Importo
---------------------------------
Pane           2    2.40     4.80
Latte          1    1.35     1.35
Parmigiano r   1    7.90     7.90
---------------------------------
```

```
TOTALE                      14.05
```

4.3 Se il cassiere sbaglia, può annullare un articolo inserendo quantità 0: l'articolo non deve comparire sullo scontrino e non conta nel totale. Usate `continue`. In fondo allo scontrino stampate anche il numero totale di pezzi.

Esempio:

```
Numero di articoli: 4
Articolo 1 - nome: Pane
Articolo 1 - prezzo: 2.40
Articolo 1 - quantità: 2
Articolo 2 - nome: Latte
Articolo 2 - prezzo: 1.35
Articolo 2 - quantità: 0
Articolo 3 - nome: Yogurt
Articolo 3 - prezzo: 0.89
Articolo 3 - quantità: 4
Articolo 4 - nome: Caffè
Articolo 4 - prezzo: 3.99
Articolo 4 - quantità: 1

Articolo     Qtà  Prezzo  Importo
--------------------------------
Pane           2    2.40     4.80
Yogurt         4    0.89     3.56
Caffè          1    3.99     3.99
--------------------------------
TOTALE                     12.35
Pezzi: 7
```

4.4 Il negozio fa una promozione 3x2: ogni tre pezzi dello stesso articolo, uno è gratis. Sotto la riga di un articolo in promozione stampate una riga con lo sconto, e in fondo quanto ha risparmiato il cliente.

Tip: i pezzi gratis sono `quantita // 3`.

Esempio (con gli stessi dati del punto 4.3; l'inserimento è omesso):

```
...
Articolo     Qtà  Prezzo  Importo
--------------------------------
Pane           2    2.40     4.80
Yogurt         4    0.89     3.56
  sconto 3x2                -0.89
Caffè          1    3.99     3.99
--------------------------------
TOTALE                     11.46
Pezzi: 7
Hai risparmiato 0.89 euro
```

4.5 **Approfondimento.** Gli specificatori di ampiezza indicano il numero *minimo* di caratteri, non il massimo. Cosa succede all'allineamento se un importo non sta in 9 caratteri, ad esempio con 10000 pezzi a 1234.50 euro? E cosa fa la precisione applicata a una stringa, come in `f"{nome:<12.12}"`?

5 Il codice a barre EAN-13

Tutti i temi insieme

Il codice a barre dei prodotti (EAN-13) è formato da 13 cifre. L'ultima è la *cifra di controllo*: si calcola dalle prime 12 e permette alla cassa di accorgersi di un errore di lettura. Si calcola così:

- si moltiplicano le 12 cifre alternativamente per 1 e per 3: la prima per 1, la seconda per 3, la terza per 1, e così via;
- si sommano i prodotti;
- la cifra di controllo è quanto manca alla somma per arrivare al multiplo di 10 successivo, e vale 0 se la somma è già un multiplo di 10. In Python: `controllo = (10 - somma % 10) % 10`.

Per esempio, per le cifre `800123456789` la somma vale 103 e la cifra di controllo è 7: il codice completo è `8001234567897`.

5.1 Partite dalla costante `CODICE = "800123456789"` e stampate una tabella con la posizione, la cifra, il peso e il prodotto di ciascuna cifra; alla fine stampate la somma, la cifra di controllo e il codice completo.

Tip: con `enumerate` avete la posizione di ogni cifra: contando da 0, le posizioni pari hanno peso 1 e quelle dispari peso 3. Ogni carattere della stringa è a sua volta una stringa: per fare i conti convertitelo con `int()`.

Esempio:

```
Pos  Cifra  Peso  Prodotto
  0      8     1         8
  1      0     3         0
  2      0     1         0
  3      1     3         3
  4      2     1         2
  5      3     3         9
  6      4     1         4
  7      5     3        15
  8      6     1         6
  9      7     3        21
 10      8     1         8
 11      9     3        27
Somma: 103
Cifra di controllo: 7
Codice completo: 8001234567897
```

5.2 Chiedete all'utente un codice completo (supponete che sia formato da 13 cifre) e verificate se è corretto: calcolate la cifra di controllo dalle prime 12 e confrontatela con la tredicesima. Non serve più stampare la tabella.

Esempio (due esecuzioni):

```
Codice EAN-13: 8001234567897
Codice valido

Codice EAN-13: 8001234567890
Codice errato: cifra di controllo 0, attesa 7
```

5.3 Sulle confezioni le cifre sono spesso separate da spazi o trattini, come in `8 001234 567897` oppure `800-1234-56789-7`. Accettate anche questi separatori, saltandoli con `continue`, e controllate che le cifre siano esattamente 13.

Tip: attenzione al peso: dipende dalla posizione della cifra tra le sole cifre, non dalla posizione nella stringa, perché i separatori non contano. La posizione di `enumerate` qui non basta: serve un contatore delle cifre lette. Un'osservazione semplifica il controllo: se si dà peso 1 anche alla tredicesima cifra (che è in posizione 12, pari), in un codice corretto la somma pesata di tutte e 13 le cifre è un multiplo di 10.

Esempio (quattro esecuzioni):

```
Codice EAN-13: 8 001234 567897
Codice valido

Codice EAN-13: 800-1234-56789-7
Codice valido

Codice EAN-13: 8001234567890
Codice errato

Codice EAN-13: 800123456789
Errore: il codice ha 12 cifre invece di 13
```

5.4 Se il codice contiene un carattere che non è né una cifra né un separatore (ad esempio una S letta al posto di un 5), interrompete subito l'analisi con `break` e segnalate il carattere e la sua posizione nella stringa: qui serve proprio la posizione di `enumerate`.

Tip: il metodo `isdigit()` dice se una stringa contiene soltanto cifre (Lab03).

Esempio (due esecuzioni):

```
Codice EAN-13: 8001234S67897
Errore: carattere 'S' in posizione 7

Codice EAN-13: 8 001234 567897
Codice valido
```

5.5 **Approfondimento.** La cifra di controllo si accorge sempre di una cifra sbagliata? E se due cifre vicine vengono scambiate, come in `8001234567897` → `8001243567897`? Provate con il vostro programma: riuscite a trovare uno scambio di due cifre vicine che non viene scoperto? Pensate alla differenza tra le due cifre scambiate.

Esercizi aggiuntivi

6 Tabella di conversione delle temperature

`range` con passo, f-string

Per convertire una temperatura da gradi Celsius a gradi Fahrenheit e a kelvin si usano le formule $F = C \times 9/5 + 32$ e $K = C + 273.15$. Scrivete un programma che stampi una tabella di conversione.

6.1 Stampate la tabella da -20 a 40 gradi Celsius, a passi di 5 gradi, con le colonne Celsius e Fahrenheit allineate a destra su 8 e 12 caratteri, con una cifra decimale. Sotto l'intestazione stampate una riga di trattini.

Tip: `range(-20, 41, 5)`: anche l'inizio può essere negativo, e la fine resta esclusa. Anche le intestazioni si possono allineare in una f-string: `f"{'Celsius':>8}"`.

Esempio (le righe intermedie sono omesse):

```
 Celsius  Fahrenheit
--------------------
   -20.0        -4.0
   -15.0         5.0
   -10.0        14.0
...
    40.0       104.0
```

6.2 Aggiungete la colonna dei kelvin, con due decimali, portate tutte le colonne a 12 caratteri e stampate sopra la tabella il titolo `Conversione di temperature`, centrato rispetto alla larghezza della tabella.

Tip: l'allineamento `^` centra un valore nello spazio disponibile.

Esempio (le righe intermedie sono omesse):

```
     Conversione di temperature
-----------------------------------
     Celsius  Fahrenheit      Kelvin
-----------------------------------
       -20.0        -4.0      253.15
       -15.0         5.0      258.15
       -10.0        14.0      263.15
...
        40.0       104.0      313.15
```

6.3 Chiedete all'utente la temperatura iniziale, quella finale e il passo (tre interi; il passo è sempre positivo) e stampate la tabella tra le due temperature, estremi compresi. Se la temperatura iniziale è maggiore di quella finale, la tabella deve essere in ordine decrescente.

Tip: con un passo negativo `range` conta all'indietro: `range(30, -1, -10)` produce 30, 20, 10, 0. La fine è esclusa anche in questo caso. Tocca al programma cambiare il segno del passo.

Esempio:

```
Da: 30
A: 0
Passo: 10
     Conversione di temperature
-----------------------------------
     Celsius  Fahrenheit      Kelvin
-----------------------------------
        30.0        86.0      303.15
        20.0        68.0      293.15
        10.0        50.0      283.15
         0.0        32.0      273.15
```

6.4 Provate a stampare la tabella da 0 a 10 gradi a passi di 2.5: cosa succede con `range(0, 11, 2.5)`? Trovate un modo per stampare la tabella anche con un passo decimale.

Tip: `range` accetta soltanto numeri interi. Contate i passi con un intero `i` e calcolate la temperatura come `inizio + i * passo`.

Esempio (prima il tentativo con `range`, poi la tabella corretta):

```
TypeError: 'float' object cannot be interpreted as an integer

     Conversione di temperature
-----------------------------------
     Celsius  Fahrenheit      Kelvin
-----------------------------------
         0.0        32.0      273.15
         2.5        36.5      275.65
         5.0        41.0      278.15
         7.5        45.5      280.65
        10.0        50.0      283.15
```

6.5 **Approfondimento.** Cosa stampa il ciclo `for c in range(30, 0, 10)`? Perché non dà errore? E `range(0, 30, 0)`?

7 Messaggi nascosti

`enumerate`, `range` con passo, `continue`, cicli annidati

Le spie nascondono i loro messaggi dentro testi che sembrano innocui. In questo esercizio li decifrate e alla fine ne nascondete uno voi.

7.1 **Acrostico.** Il messaggio è formato dalle iniziali delle parole di una frase. Data la costante

```
FRASE = "svegliati velocemente e guarda laggiù in alto"
```

stampate il messaggio nascosto, in maiuscolo.

Tip: un carattere è l'iniziale di una parola se non è uno spazio e se è il primo carattere della frase, oppure se il carattere che lo precede è uno spazio. Per guardare il carattere precedente serve la posizione: usate `enumerate`. Attenzione all'ordine delle condizioni: per `i = 0`, che carattere è `FRASE[i - 1]`?

Esempio:

```
Messaggio: SVEGLIA
```

7.2 **Una lettera ogni k.** Un'altra spia ha intercettato il testo

```
TESTO = "kebmuibcbrdllsibqogbtcnencchca"
```

Il messaggio è formato da una lettera ogni k: la k-esima, la $2k$-esima, e così via. Chiedete k e stampate il messaggio nascosto, usando un `range` con passo sulle posizioni della stringa. Il messaggio vero si legge con $k = 3$.

Tip: la k-esima lettera ha indice `k - 1`.

Esempio (due esecuzioni):

```
Una lettera ogni: 3
Messaggio: biblioteca

Una lettera ogni: 2
Messaggio: emicrlsbobcecha
```

7.3 La spia ora inserisce degli spazi a caso, perché il testo sembri fatto di parole:

```
TESTO = "orybooq isgfn zqpau t zrfd oi je c or z wtubtrm oue"
```

Gli spazi non contano: il messaggio è formato da una lettera ogni k, contando soltanto le lettere. Con $k = 4$ il programma del punto 7.2 stampa `b fq roeo b`: modificatelo saltando gli spazi con `continue`.

Tip: la posizione nella stringa non serve più, perché conta anche gli spazi: usate un contatore delle lettere.

Esempio:

```
Una lettera ogni: 4
Messaggio: binariotre
```

7.4 Ora tocca a voi. Chiedete un messaggio segreto e il valore di k, e costruite un testo cifrato in cui prima di ogni lettera del messaggio ci sono $k-1$ lettere minuscole casuali. Verificate il risultato decifrandolo con il metodo del punto 7.2.

Tip: servono due cicli annidati: quello esterno scorre le lettere del messaggio, quello interno aggiunge le $k - 1$ lettere casuali. Una lettera minuscola casuale si ottiene con `chr(random.randint(ord("a"), ord("z")))`: `ord` e `chr` li avete usati nel Lab02 (esercizio B.8). Le lettere sono casuali, quindi il vostro testo cifrato sarà diverso da quello dell'esempio.

Esempio:

```
Messaggio segreto: pizzeria
Una lettera ogni: 3
Testo cifrato: dkpqqiudzhtztrenzrsrixya
Verifica: pizzeria
```

7.5 **Approfondimento.** Nel Lab02 (esercizio A.6) avete visto lo slicing con passo, come in `s[::2]`. Riuscite a risolvere il punto 7.2 senza ciclo, con un solo slicing?

8 Sistemi di numerazione

f-string con i tipi `b`, `o` e `x`, `for` su una stringa, `break`

Noi scriviamo i numeri in base 10, il computer li memorizza in base 2. Le f-string sanno stampare un intero anche in base 2, 8 e 16, con i tipi `b` (binario), `o` (ottale) e `x` o `X` (esadecimale, con le lettere minuscole o maiuscole).

8.1 Stampate una tabella con i numeri da 0 a 16 scritti in base 10, 2, 8 e 16. Le cifre binarie devono essere sempre 8, come i bit di un byte, con gli zeri a sinistra.

Tip: come per gli orari dell'esercizio 1, uno 0 davanti all'ampiezza riempie con gli zeri: `f"{5:08b}"` dà `00000101`.

Esempio (le righe intermedie sono omesse):

```
 dec       bin  oct  hex
   0  00000000    0    0
   1  00000001    1    1
   2  00000010    2    2
...
  10  00001010   12    A
  11  00001011   13    B
  12  00001100   14    C
  13  00001101   15    D
  14  00001110   16    E
  15  00001111   17    F
  16  00010000   20   10
```

8.2 Il contrario: chiedete un numero binario e calcolatene il valore in base 10. Scorrete le cifre da sinistra a destra: a ogni cifra il valore calcolato fino a quel momento si moltiplica per 2 e gli si aggiunge la cifra (`valore = valore * 2 + cifra`). Stampate il valore a ogni passo, per vederlo crescere.

Tip: è lo stesso ragionamento che si fa in base 10: leggendo 1729 da sinistra si ha 1, poi $17 = 1 \times 10 + 7$, poi $172 = 17 \times 10 + 2$, infine $1729 = 172 \times 10 + 9$.

Esempio:

```
Numero binario: 1101
1 -> 1
1 -> 3
0 -> 6
1 -> 13
Valore decimale: 13
```

8.3 Controllate che la stringa contenga soltanto le cifre 0 e 1: al primo carattere non valido stampate un messaggio con la sua posizione e interrompete il ciclo con `break`. Se il numero è valido, stampate soltanto il risultato finale.

Esempio (due esecuzioni):

```
Numero binario: 101101
Valore decimale: 45

Numero binario: 10210
Errore: '2' in posizione 2 non è una cifra binaria
```

8.4 Estendete il programma a una base qualsiasi tra 2 e 10, chiesta all'utente: le cifre ammesse sono quelle da 0 a `base - 1`, e a ogni passo il valore si moltiplica per la base.

Esempio (due esecuzioni):

```
Base (da 2 a 10): 8
Numero in base 8: 777
Valore decimale: 511

Base (da 2 a 10): 8
Numero in base 8: 1238
Errore: '8' in posizione 3 non è valido in base 8
```

8.5 **Approfondimento.** In base 16 le cifre da 10 a 15 si scrivono con le lettere da A a F. Come potete calcolare il valore di una cifra che è una lettera? Ripensate a come avete calcolato la posizione di una lettera nell'alfabeto nel cifrario di Cesare (Lab02, esercizio B.8). Per verificare i risultati potete usare `int("FF", 16)`, che vale 255.

9 I calci di rigore

Tutti i temi insieme

Una partita finita in parità si decide ai calci di rigore. Ogni squadra tira 5 rigori e le squadre si alternano: in ogni turno tira prima la squadra A, poi la squadra B. Vince chi segna più gol. Simulate una serie di rigori in cui ogni tiro va a segno con probabilità del 75%. Il programma si costruisce in punti: ciascuno parte dal programma del precedente.

Tip: `random.random()` restituisce un numero casuale tra 0 (compreso) e 1 (escluso), come nel metodo Monte Carlo visto a lezione: la condizione `random.random() < 0.75` è vera, in media, 3 volte su 4. Poiché i tiri sono casuali, i vostri risultati saranno diversi da quelli degli esempi.

9.1 Simulate i 5 turni con un ciclo `for`. A ogni turno stampate l'esito dei due tiri (`gol` o `parato`) e il punteggio; alla fine stampate chi ha vinto, oppure il pareggio. Definite come costanti il numero di rigori e la probabilità di segnare.

Tip: allineate gli esiti con una f-string, ad esempio `{esito_a:<7}`, in modo che i punteggi restino in colonna.

Esempio:

```
Turno 1:  A gol     B parato  1-0
Turno 2:  A parato  B gol     1-1
Turno 3:  A gol     B gol     2-2
Turno 4:  A gol     B parato  3-2
Turno 5:  A gol     B gol     4-3
Vince A 4-3
```

9.2 **Fine anticipata.** La serie finisce appena una squadra non può più essere raggiunta, anche se mancano dei tiri, e anche a metà di un turno. Ad esempio, se dopo 4 rigori della squadra

A e 3 della squadra B il punteggio è 1-3, alla squadra A resta un solo tiro: al massimo arriva a 2, e la serie è finita. Stampate ora un tiro per riga e, alla fine, quanti rigori non è stato necessario tirare.

Tip: conviene un ciclo interno su una stringa: `for squadra in "AB"` fa tirare prima A, poi B. Contate i tiri di ciascuna squadra: dopo ogni tiro, una squadra è irraggiungibile se il suo vantaggio è maggiore del numero di tiri che restano all'altra. Attenzione: `break` esce soltanto dal ciclo interno (esercizio 3.3); per uscire anche da quello esterno serve una variabile booleana.

Esempio (due esecuzioni):

```
Turno 1 - A: parato  0-0
Turno 1 - B: gol     0-1
Turno 2 - A: gol     1-1
Turno 2 - B: gol     1-2
Turno 3 - A: parato  1-2
Turno 3 - B: gol     1-3
Turno 4 - A: parato  1-3
Serie decisa in anticipo (rigori non tirati: 3)
Vince B 3-1

Turno 1 - A: gol     1-0
Turno 1 - B: parato  1-0
Turno 2 - A: parato  1-0
Turno 2 - B: gol     1-1
Turno 3 - A: gol     2-1
Turno 3 - B: gol     2-2
Turno 4 - A: gol     3-2
Turno 4 - B: parato  3-2
Turno 5 - A: gol     4-2
Serie decisa in anticipo (rigori non tirati: 1)
Vince A 4-2
```

9.3 **Morte improvvisa.** Se dopo i 5 turni il punteggio è pari, si continua a oltranza un turno alla volta: tirano entrambe le squadre, e la serie finisce al primo turno in cui una segna e l'altra no. Il numero di turni non è noto in anticipo: che tipo di ciclo serve?

Esempio:

```
Turno 1 - A: gol     1-0
Turno 1 - B: gol     1-1
Turno 2 - A: parato  1-1
Turno 2 - B: parato  1-1
Turno 3 - A: gol     2-1
Turno 3 - B: parato  2-1
Turno 4 - A: gol     3-1
Turno 4 - B: gol     3-2
Turno 5 - A: parato  3-2
Turno 5 - B: gol     3-3
Turno 6 - A: parato  3-3
Turno 6 - B: gol     3-4
Serie decisa alla morte improvvisa, al turno 6
Vince B 4-3
```

9.4 **Statistiche.** Togliete le stampe dei singoli tiri e simulate 10000 serie di rigori, con un altro ciclo esterno a tutto il resto. Stampate la percentuale di vittorie di ciascuna squadra,

quella delle serie decise in anticipo e quella delle serie arrivate alla morte improvvisa, con una cifra decimale.

Esempio (i vostri numeri saranno leggermente diversi):

```
Serie simulate: 10000
Vittorie di A:          50.4%
Vittorie di B:          49.6%
Decise in anticipo:     44.4%
Alla morte improvvisa:  29.5%
```

9.5 **Approfondimento.** Tirare per primi è un vantaggio? Nel vostro modello le due squadre hanno la stessa probabilità di segnare: quale percentuale di vittorie vi aspettate per la squadra A? Provate poi a dare alla squadra B una probabilità del 70%: quanto cambia la percentuale di vittorie?